

Московский авиационный институт (национальный исследовательский университет)
Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра № 806 «Вычислительная математика и программирование»

Разработка и исследование пулов потоков с механизмом `work-stealing` на языке C++

Выпускная квалификационная работа студента СпецВО-магистратуры

Студент группы М8О-214СВ-24: Спиридонов Кирилл Анатольевич
Научный руководитель: д.ф.-м.н., доцент Морозов Александр Юрьевич

Москва — 2026



- В начале 2000-х годов рост тактовой частоты процессоров достиг физических пределов — индустрия перешла к многоядерным архитектурам
- Флагманские серверные процессоры содержат **сотни ядер**; параллелизм стал повседневной инженерной реальностью
- Классический пул потоков с **единой глобальной очередью** плохо масштабируется: очередь превращается в узкое место при большом числе потоков
- Стандартная библиотека **C++17** не **предоставляет** готовой реализации пула потоков

Проблема

Отсутствует реализация пула потоков с механизмом work-stealing, построенная исключительно на стандарте C++17 без внешних зависимостей и подкреплённая систематическим экспериментальным исследованием



Цель — разработать библиотеку на языке C++, реализующую пул потоков с механизмом work-stealing, и экспериментально исследовать её производительность в сравнении с классическим пулом потоков

Задачи:

- Проанализировать существующие подходы и реализации пулов потоков
- Спроектировать архитектуру библиотеки с единым интерфейсом для двух реализаций
- Реализовать SimpleThreadPool и WorkStealingPool без внешних зависимостей
- Верифицировать корректность посредством автоматических тестов и Thread Sanitizer
- Провести сравнительное исследование производительности и сформулировать практические рекомендации



Дано:

- Многоядерная вычислительная система
- Поток задач с неизвестной заранее структурой нагрузки
- Теоретическая оценка Блюмофе–Лейзерсона:

$$T_P \leq \frac{T_1}{P} + O(T_\infty)$$

- Алгоритм Chase–Lev для двусторонней очереди

Требования к решению:

- Язык — C++17, без внешних зависимостей
- Единый интерфейс `submit()` для обеих реализаций
- Корректность при конкурентном доступе (отсутствие гонок данных)

Необходимо:

Библиотека с двумя взаимозаменяемыми реализациями пула потоков и экспериментальное сравнение их характеристик



Модель Блюмофе–Лейзерсона:

- Параллельное вычисление — ориентированный ациклический граф
- T_1 — работа (однопроцессорное время)
- T_∞ — глубина (критический путь)
- Гарантия жадного work-stealing:

$$T_P \leq \frac{T_1}{P} + O(T_\infty)$$

Закон Амдала:

$$S_P \leq \frac{1}{\alpha + \frac{1-\alpha}{P}}$$

Принцип работы deque:

Владелец (LIFO, нижний конец)

`push(task)` `pop(task)`

В типичном случае — без конкуренции

Вор (FIFO, верхний конец)

`steal(task)`

Забирает «старые» крупные задачи

Асимметрия обеспечивает низкую конкуренцию и хорошую локальность данных



Библиотека состоит из трёх независимых компонентов:

WorkStealingDeque

Двусторонняя очередь.
Асимметричный доступ
владельца и воров.
Защита: `std::mutex`

SimpleThreadPool

Классический подход.
Единая очередь +
`condition_variable`

WorkStealingPool

Локальные очереди.
Алгоритм кражи задач:
случайная жертва через
`mt19937`

Стек технологий: C++17, CMake $\geq$ 3.14, Google Test, Google Benchmark.



Единый шаблонный интерфейс:

```
template<typename F, typename... Args>  
auto submit(F&& f, Args&&... args) -> std::future<invoke_result_t<F,Args...>>
```

Обеспечивает взаимозаменяемость реализаций пулов для корректного тестирования.

Управление жизненным циклом (RAII):

- **Конструктор:** автоматически создаёт рабочие потоки (по умолчанию используется `std::thread::hardware_concurrency()`).
- **Деструктор:** реализует паттерн *graceful shutdown* — безопасно дожидается завершения всех находящихся в очереди задач.
- **Ограничения семантики:** объекты пулов объявлены не копируемыми и перемещаемыми.



Архитектура:

- Единая глобальная очередь задач `task_queue_`
- Фиксированный пул рабочих потоков `workers_`
- Синхронизация: `std::mutex` + `std::condition_variable`
- Потоки-блокировщики: все воркеры конкурируют за один мьютекс

Жизненный цикл задачи:

- 1 `submit()` упаковывает задачу в `std::packaged_task`
- 2 Задача помещается в очередь под защитой мьютекса
- 3 `cv_.notify_one()` будит одного из ожидающих воркеров
- 4 Воркер забирает задачу (FIFO) и выполняет её

Преимущества

- Простота реализации
- Минимум аллокаций
- Предсказуемый порядок выполнения

Ограничения

- При увеличении числа потоков возрастает конкуренция за мьютекс



Структура пула:

- Каждый поток имеет **локальную очередь** (`work_stealing_queue`)
- Глобальная очередь — точка входа для внешних задач

Алгоритм `submit()`:

- Вызов **из воркера** — задача идёт в `local_work_queue`
- Вызов **извне** — задача идёт в глобальную очередь

Рабочий цикл потока:

- 1 Взять задачу из **локальной** очереди (`try_pop`)
- 2 Если пусто — взять из **глобальной** очереди (`try_pop`)
- 3 Если пусто — украсть из очереди **соседнего** потока (`try_steal`)
- 4 Если везде пусто — `yield()` и повторить



28 автоматических тестов (Google Test): Ключевые тесты:

Набор тестов	Тестов
WorkStealingDeque	12
SimpleThreadPool	6
WorkStealingPool	10
Итого	28

- `SingleElementRaceCondition` — 10 000 итераций: владелец и вор одновременно претендуют на последний элемент; задача выполняется ровно один раз
- `DestructorWaitsPending` — деструктор дожидается всех задач
- `HeavyLoadAllTasks` — 100 000 задач под нагрузкой

Результат

Все 28 тестов пройдены. Thread Sanitizer не обнаружил ни одной гонки данных. Сборка с `-fsanitize=thread` интегрирована в CMake.



Тестовая машина:

Параметр	Значение
Логических ядер	16
Физических ядер	8
Тактовая частота	5102,71 МГц
Кэш L1	32 КиБ × 8
Кэш L2	1024 КиБ × 8
Кэш L3	16384 КиБ × 1
ОС	Linux
Компилятор	Clang, C++17
Оптимизация	-O2

Два сценария нагрузки:

Сценарий 1: много мелких задач

10 000 задач, каждая — атомарный инкремент (≈ 5 нс).

Измеряет накладные расходы планировщика.

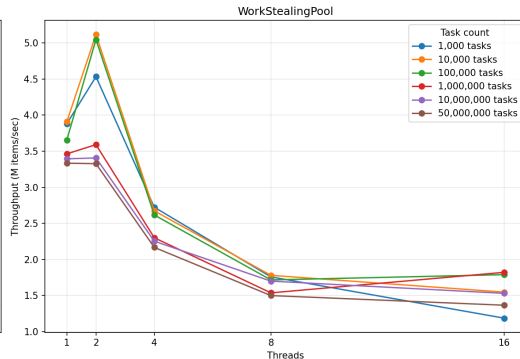
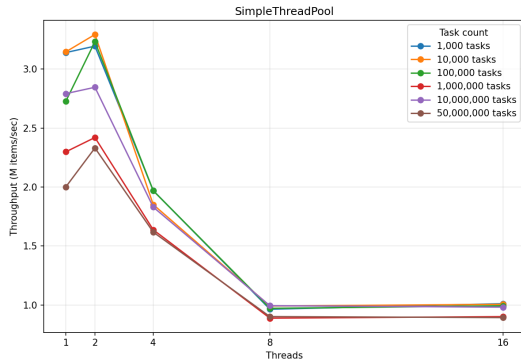
Сценарий 2: неравномерная нагрузка

10 000 задач: половина — атомарный инкремент,
половина — `sleep_for(200 мкс)`.

Целевой сценарий для work-stealing.

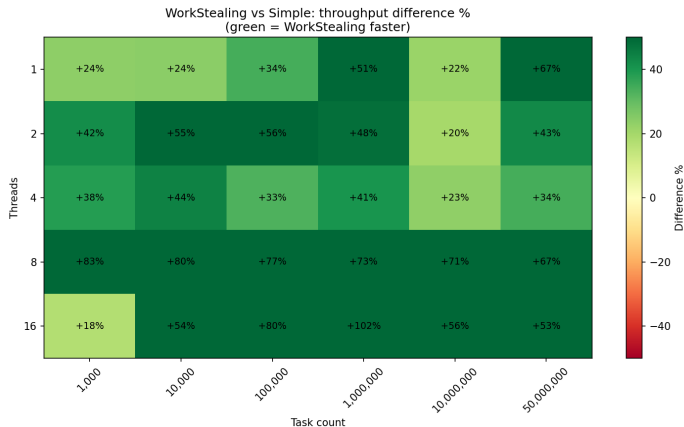


Throughput vs Threads (items/sec)



Зависимость пропускной способности пулов (млн. задач/с) от количества рабочих потоков. График демонстрирует деградацию производительности (lock contention) при увеличении числа потоков из-за высокой конкуренции за глобальную очередь.

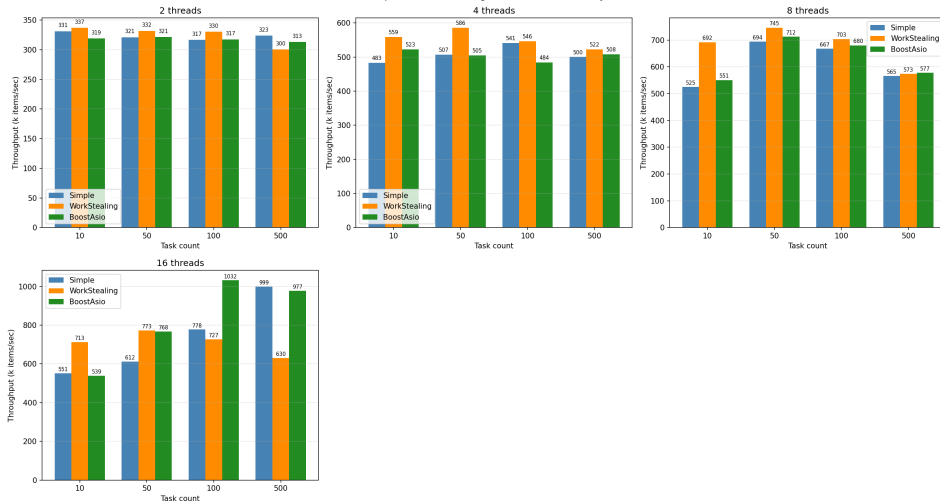




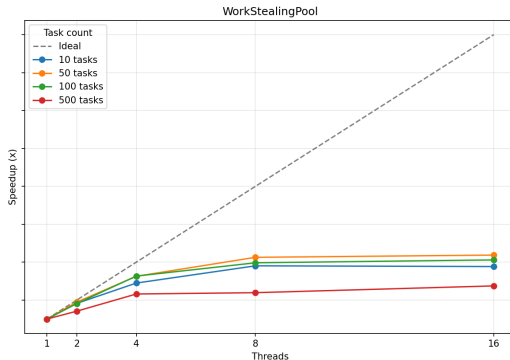
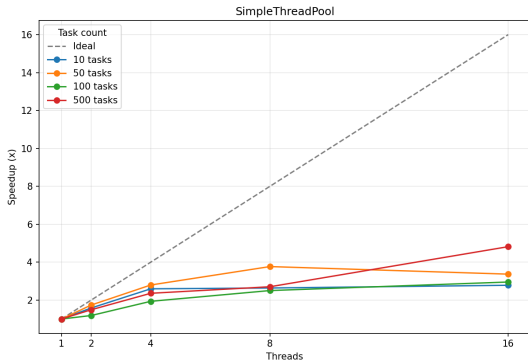
Сравнение WorkStealing и Simple Pool: ускорение WorkStealing относительно Simple (лёгкие задачи).



CPU-Bound: Simple vs WorkStealing vs BoostAsio по числу потоков



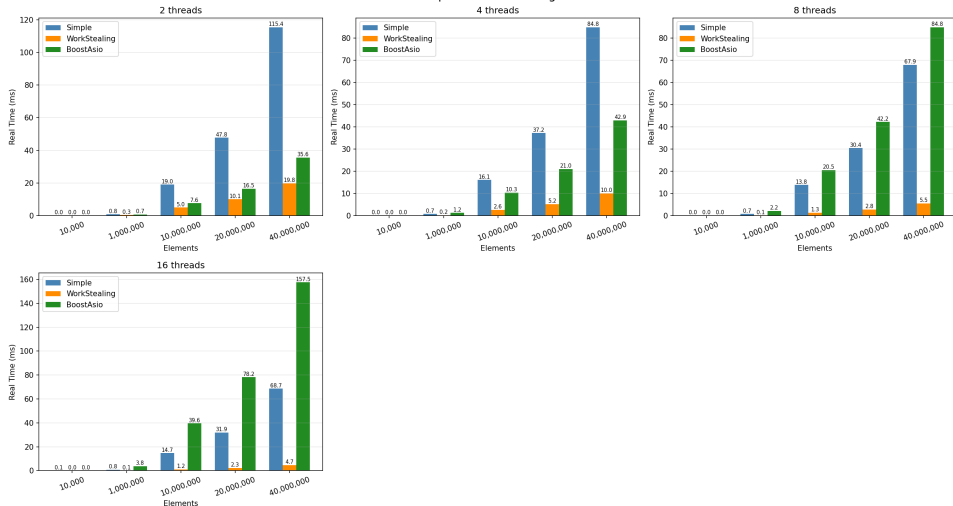
CPU-Bound: Speedup vs Threads (относительно 1 потока)



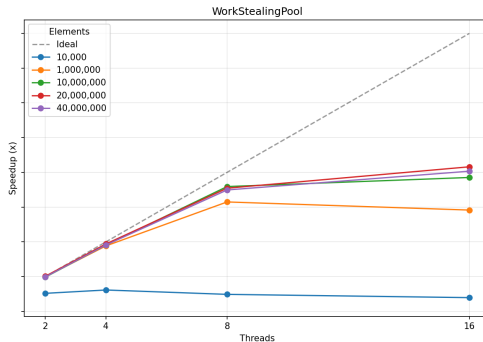
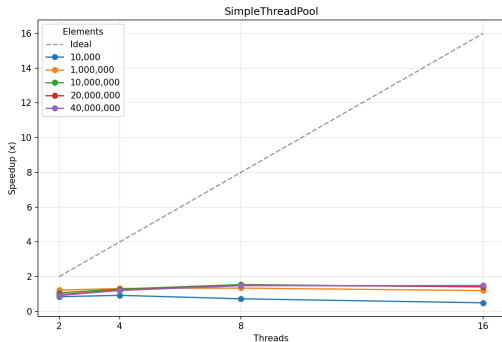
Ускорение пулов потоков относительно однопоточного выполнения. Пунктир — идеальное масштабирование.



Nested Tasks: Simple vs WorkStealing vs BoostAsio

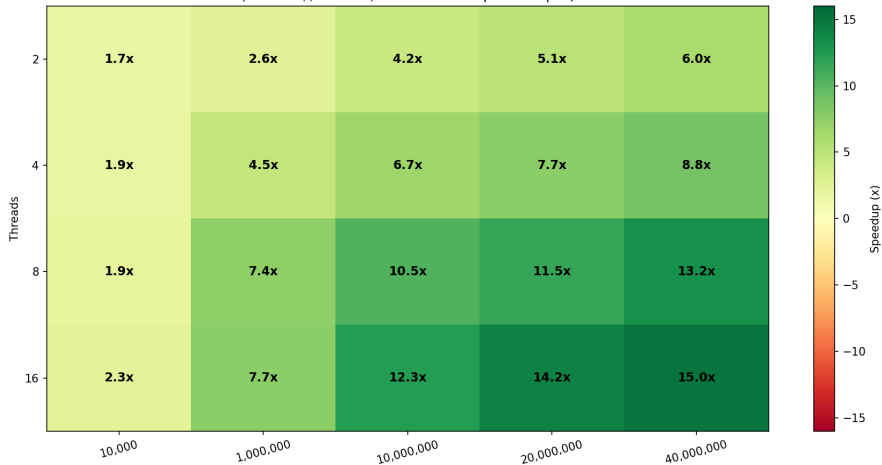


Nested Tasks: Speedup vs Threads (относительно Sequential)



WorkStealing vs Simple: два способа измерить разницу

Speedup (во сколько раз WS быстрее Simple)
(1.0x = одинаково, 15x = WS быстрее в 15 раз)



Лёгкие задачи ($i * i$):

- WorkStealing быстрее на **20–78%** при любом числе потоков
- Причина: `yield()` дешевле, чем `condition_variable` — поток не засыпает и сразу берёт следующую задачу
- Оба пула деградируют при 4+ потоках — задача **дешевле** накладных расходов на синхронизацию
- Оптимум: **2 потока**

Тяжёлые задачи ($\sin/\cos/\sqrt{}$):

- Оба пула показывают **схожие результаты** — разница в пределах 5%
- Overhead на синхронизацию незаметен: задача ≈ 0.1 мс, overhead $\approx$ несколько мкс
- Оба масштабируются до **8 физических ядер** с ускорением $\approx 6.5x$
- При 16 потоках прирост минимален — эффект Hyper-Threading



Рекурсивный `parallel_sum`:

- WorkStealing быстрее в **8–16 раз** при 8–16 потоках
- Simple Pool **деградирует** при росте потоков — при 2 потоках медленнее, чем при 1 потоке
- Причина: Simple Pool блокируется на `future.get()` — поток простаивает
- WorkStealing вызывает `run_pending_task()` — поток работает пока ждёт

Ключевой результат

Для рекурсивных алгоритмов divide-and-conquer WorkStealing Pool быстрее в **8–16 раз**

Почему Simple Pool деградирует?

Все потоки блокируются на разных уровнях рекурсии и ждут друг друга —



Причины кратного превосходства Work-Stealing пула:

- 1 Изоляция блокировок:**
Отказ от глобального мьютекса при рекурсии.
Подзадачи создаются в `thread_local` очередях с нулевой конкуренцией (Contention).
- 2 Снижение латентности:**
Использование активного ожидания вместо `condition_variable` исключает накладные расходы ОС на засыпание и пробуждение потоков.
- 3 Кэш-локальность (LIFO):**
Обработка собственных задач с конца очереди сохраняет актуальные данные в L1/L2 кэше процессора.

Инженерные рекомендации:

WorkStealingPool

Применение: Алгоритмы «Разделяй и властвуй», обход графов, рендеринг.
Особенность: Обеспечивает максимальную пропускную способность, но потребляет 100% CPU в простое.

SimpleThreadPool

Применение: I/O операции, сетевые запросы, фоновые долгие задачи.
Особенность: Энергоэффективен. Потоки глубоко засыпают при отсутствии работы, освобождая ресурсы системы.



- 1 **Реализована библиотека на чистом C++17** (без внешних зависимостей). Спроектированы SimpleThreadPool и WorkStealingPool с единым future-совместимым интерфейсом submit().
- 2 **Обеспечена многопоточная надежность:** код успешно прошел стресс-тестирование и проверки Thread Sanitizer (гонки данных не выявлены). Решена проблема мертвой блокировки (starvation deadlock) при рекурсии.
- 3 **Показано превосходство Work-Stealing:** на вложенных задачах (Divide & Conquer) алгоритм продемонстрировал линейное масштабирование, опередив базовую реализацию в **16 раз** (на 16 потоках) за счет устранения конкуренции за мьютекс.
- 4 **Установлен фундаментальный компромисс:** профилирование доказало, что на CPU-bound задачах архитектура пула не влияет на скорость. Установлено, что высокая пропускная способность Work-Stealing достигается ценой 100% утилизации процессора (активное ожидание).

Перспективы развития

Отказ от блокирующих мьютексов в пользу Lock-Free структуры данных.



Полный исходный код размещён в открытом репозитории GitHub.

